C N 2025-10-29

Risoluzione di sistemi lineari.

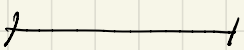
⇒ Riguardare l'appendice A1 del libro.

Risolvere

(1) $A\underline{x} = \underline{b}$, $A \in \mathbb{R}^{n\times n}$, $\det(A) \neq 0$, $\underline{x}, \underline{b} \in \mathbb{R}^n$.

Avevamo cominciato a vedere i casi più semplici in cui (1) è facilmente risolvibile.

- A diagonale, $O(n)$ memoria ed operazioni;
- A triangolare,
- A ortogonale.



Prodotto matrice-vettore.

$$\mathbb{R}^{n\times n} \ni A = \begin{bmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & & \vdots \\ a_{n1} & \cdots & a_{nn} \end{bmatrix} = \begin{bmatrix} \underline{r}_1^T \\ \vdots \\ \underline{r}_n^T \end{bmatrix} = [\underline{c}_1, \dots, \underline{c}_n],$$

con:

$\underline{r}_i^T = (a_{i1}, \dots, a_{in})$, $i = 1, \dots, n$, i-esima riga di A,

$\underline{c}_j = \begin{bmatrix} a_{1j} \\ \vdots \\ a_{nj} \end{bmatrix}$, $j = 1, \dots, n$, j-esima colonna di A.

Se definiamo

$$\underline{e}_i = \begin{bmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix} \leftarrow i \in \mathbb{R}^n, \quad i = 1, \dots, n,$$

allora: $\underline{e}_i^T A = \underline{r}_i^T$, è la i-esima riga della matrice A;

$A \underline{e}_j = \underline{c}_j$, è la j-esima colonna di A;

$\underline{e}_i^T A \underline{e}_j = a_{ij}$, è l'elemento (i,j) della matrice A.

Inoltre:

$$\mathbb{R}^{n \times n} \ni \underline{e}_i \underline{e}_i^T = \begin{bmatrix} & & \\ & 1 & \leftarrow i \\ & \vdots & \\ & \downarrow & \end{bmatrix}$$

da questo segue che:

$$\sum_{j=1}^{n} \underline{e}_j \underline{e}_j^T = I \in \mathbb{R}^{n \times n},$$ la matrice identità.

A questo punto, vediamo in quanti modi,

se $\underline{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}$, e $\underline{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}$, posso calcolare il vettore

$$\underline{y} = A \cdot \underline{x}$$

Abbiamo, essenzialmente, 2 modi di calcolare $\underline{y}$. Il primo è il classico prodotto riga per colonna:

(2) $$y_i = \underline{r}_i^T \underline{x} \quad , i = 1, \dots, n.$$ (scal, o prodotto scalare)

Un modo, algebricamente equivalente, è il seguente:

$$\underline{y} = A\underline{x} = A \cdot I \cdot \underline{x}$$

$$= A \sum_{j=1}^{n} \underline{e}_j \underline{e}_j^T \underline{x}$$

$$= \sum_{j=1}^{n} (A\underline{e}_j)(\underline{e}_j^T \underline{x})$$

(3) $$= \sum_{j=1}^{n} \underline{c}_j \cdot x_j$$ (n axpy, ovvero operazioni

del tipo

$\underline{v} = \underline{\alpha} x + \underline{y}$)

Vediamo di codificare (2) e (3)

x e y sono i vettori, e a è nxn.

$y \leftarrow 0$ (inizializzazione)

(2)

```
for i = 1:n
    for j = 1:n
        y(i) = y(i) + a(i,j) * x(j)
    end
end
```

n scal

(3)

```
for j = 1:n
    for i = 1:n
        y(i) = y(i) + a(i,j) * x(j)
    end
end
```

n axpy

Il costo, in termini di operazioni, è

lo stesso, ovvero $2n^2$ flops (floating point operations)

OSS Se $A \in \mathbb{R}^{m \times n}$, avremmo avuto $2mn$ flops.

La scelta tra (2) o (3) dipende dalla modalità di accesso ai dati. Infatti, una matrice è memorizzata come un vettore di dimensione n^2 (mn nel caso $A \in \mathbb{R}^{m \times n}$), o columnwise o rowwise, ovvero, per colonne

$$A \to \begin{bmatrix} \underline{c}_1 \\ \underline{c}_2 \\ \vdots \\ \underline{c}_n \end{bmatrix},$$ o per righe

$$A \to [\underline{r}_1^T \ \underline{r}_2^T \ \dots \ \underline{r}_n^T].$$

Chiaramente, per sfruttare la località dei dati, utilizzeremo (2) se A è memorizzata per righe, ovvero (3)

se A è memorizzata per colonne.
Questo dipenderà dal linguaggio di programmazione che stiamo utilizzando.

Ritorniamo ad esaminare il caso di sistemi lineari "semplici" e, in particolare, il caso di matrici triangolari.

Def. Diremo che $A = (a_{ij}) \in \mathbb{R}^{n \times n}$, è triangolare:

- inferiore, se $a_{ij} = 0, \quad j > i$;
- superiore, se $a_{ij} = 0, \quad i > j$.

Es:

Data $$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{33} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

se A fosse triangolare inferiore, allora:

$$A = \begin{bmatrix} a_{11} & & \\ a_{21} & a_{22} & \\ a_{31} & a_{32} & a_{33} \end{bmatrix}.$$

Viceversa, se A fosse triangolare superiore, allora:

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ & a_{22} & a_{33} \\ & & a_{33} \end{bmatrix}.$$

<u>OSS</u> Se A fosse al contempo triangolare inferiore e triangolare superiore, allora:

$$A = \begin{bmatrix} a_{11} & & \\ & a_{22} & \\ & & a_{33} \end{bmatrix},$$

ovvero A è diagonale.

<u>N.B.</u> Come per matrici diagonali, se $A = (a_{ij}) \in \mathbb{R}^{n \times n}$, allora

$$\det(A) = \prod_{i=1}^{n} a_{ii}.$$

Ne consegue che $\det(A) \neq 0$ se e solo se $a_{ii} \neq 0$, $i = 1, \dots, n$.

Esaminiamo il caso in cui A è triangolare inferiore, poiché il caso triangolare superiore è analogo. Pertanto:

$$A\underline{x} = \begin{bmatrix} a_{11} & & & & \\ a_{21} & a_{22} & & & \\ a_{31} & a_{32} & a_{33} & & \\ \vdots & \vdots & & \ddots & \\ a_{n1} & a_{n2} & \cdots & & a_{nn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_n \end{bmatrix} = \underline{b}$$

ovvero:

$$\begin{cases} a_{11}x_1 = b_1 \\ a_{21}x_1 + a_{22}x_2 = b_2 \\ a_{31}x_1 + a_{32}x_2 + a_{33}x_3 = b_3 \\ \vdots \\ a_{n1}x_1 + a_{n2}x_2 + a_{n3}x_3 + \cdots + a_{nn}x_n = b_n \end{cases}$$

Possiamo quindi risolvere mediante sostituzioni successive in avanti:

$$x_i = \frac{b_i - \sum_{j=1}^{i-1} a_{ij}x_j}{a_{ii}}, \quad i = 1, \dots, n, \tag{4}$$

con la convenzione $\sum_{j=1}^{0} = 0$.

Dalla (h) otteniamo che:

1) l'algoritmo risolutivo è ben definito se e solo se $a_{ii} \neq 0 \quad \forall i = 1, \dots, n$

$\iff \det(A) \neq 0$, come abbiamo assunto;

2) il numero di operazioni all'iterazione i-esima è $2i-1$ flops, per un totale di

$$\sum_{i=1}^{n} (2i-1) = 2 \sum_{i=1}^{n} i - n$$

$$= 2 \cdot \frac{n(n+1)}{2} - n = n^2 \text{ flops.}$$

Anche la memoria richiesta è $\approx \frac{n^2}{2}$ posizioni di memoria.

Pertanto la complessità è $O(n^2)$, sia in termini di memoria, che di flops.

Scriviamo uno pseudo-codice per (h), in cui supponiamo di avere i vettori

x e b e la matrice a nxn.

inizializziamo $\underline{x} \leftarrow \underline{b}$.

(5)

```
for i = 1:n
    for j = 1:i-1
        x(i) = x(i) - a(i,j) * x(j)        scal
    end
    x(i) = x(i) / a(i,i)
end
```

In questa implementazione, si ha un accesso per righe agli elementi di a.

Si può ottenere un algoritmo risolutivo equivalente, con un accesso per colonna, come segue ($\underline{x} \leftarrow \underline{b}$, iniz.)

(6)

```
for j = 1:n
    x(j) = x(j) / a(j,j)
    for i = j+1:n
        x(i) = x(i) - a(i,j) * x(j)        axpy
    end
end
```

oss Osserviamo che (5) e (6), sebbene algebricamente equivalenti, in aritmetica finita producono generalmente risultati non uguali.